Object Oriented Programming I

September 18, 2024

Content

friend Functions and friend Classes

Using the this Pointer

static Class Members

Introduction

- One of the important concepts of Object Oriented Programming (OOP) is encapsulation (data hiding).
- For example, a non-member function cannot access an object's private or protected data.
- But, sometimes this restriction may force programmer to write long and complex codes.
- So, there is mechanism built in C++ programming to access private or protected data from non-member functions.
- This is done using a **friend function** or/and a **friend class**.

Friend Functions

- If a function is defined as a friend function then, the private and protected data of a class can be accessed using the function.
- The complier knows a given function is a friend function by the use of the **keyword friend**.
- **For accessing the data, the declaration of a friend function should be made inside the body of the class (can be anywhere inside class either in private or public section) starting with keyword friend.**

Friend Functions

```
10  #include <iostream>
11  using namespace std;
12
13  class Count
14  {
15      friend void setX( Count &, int);
16
17  public:
18      /// Constructor
19      Count()
20          :x(0)
21      {
22          // Empty Body
23      }
24      /// Prints data member x
25      void print() const
26      {
27          cout << x << endl;
28      }
29
30  private:
31      int x;
32  };
33
34  // sets the value of private data member x
35  void setX( Count &c, int value )
36  {
37      c.x= value;
38  }
39
40  main()
41  {
42      Count counter;  // create Count Object
43
44      cout << "counter.x after instantination: ";
45      counter.print();
46
47      setX(counter,8);    // set x using a friend function
48      cout << "counter.x after call to setX friend function: ";
49      counter.print();
50  }
```

- The friend declaration (line 15) appears first (by convention) in the class definition, even before public member functions are declared.
- Again, this friend declaration **can appear anywhere in the class**.
- Function setX (lines 35-38) is a C-style, stand-alone function and it isn't a member function of class Count.
- If you remove the friend declaration in line 15, you'll receive error messages indicating that function setX cannot modify class Count's private data member x.

Friend Functions

```
10  #include <iostream>
11  using namespace std;
12
13  class Count
14  {
15      friend void setX( Count &, int);
16
17  public:
18      /// Constructor
19      Count()
20          :x(0)
21      {
22          // Empty Body
23      }
24      /// Prints data member x
25      void print() const
26      {
27          cout << x << endl;
28      }
29
30  private:
31      int x;
32  };
33
34  // sets the value of private data member x
35  void setX( Count &c, int value )
36  {
37      c.x= value;
38  }
39
40  main()
41  {
42      Count counter;  // create Count Object
43
44      cout << "counter.x after instantination: ";
45      counter.print();
46
47      setX(counter,8);    // set x using a friend function
48      cout << "counter.x after call to setX friend function: ";
49      counter.print();
50  }
```

- Function setX (lines 35-38) is a C-style, stand-alone function and it isn't a member function of class Count.
- For this reason, when setX is invoked for object counter,line 47 passes counter as an argument to setX rather than using a handle (such as the name of the object) to call the function, as in

  counter.setX(8);
- Notice that the friend keyword is not used in the header of the setX function (in line 35).

Friend Functions

```
10  #include <iostream>
11  using namespace std;
12
13  class Box {
14
15  public:
16        friend void printWidth(Box b);
17        void setWidth( double wid );
18  private:
19      double width;
20  };
21
22  // Member function definition
23  void Box::setWidth( double wid ) {
24     width = wid;
25  }
26  // Note: printWidth() is not a member function of any class.
27  void printWidth( Box b ) {
28     /* Because printWidth() is a friend of Box, it can
29     directly access any member of this class */
30     cout << "Width of box : " << b.width <<endl;
31  }
32  // Main function for the program
33  main() {
34
35     Box box;
36     // set box width without member function
37     box.setWidth(10.0);
38     // Use friend function to print the wdith.
39     printWidth( box );
40  }
```

- Notice that friend function is declared in public access specifier. Again, friend function declarations **can appear anywhere in the class including under private access specifier**.
- Notice that in line 27 printWidth friend function does not contain Box::, because it is not a member function of the class.
- Notice that in line 39 printWidth friend function calls with Box object (in this example box).

Friend Functions

```
10  #include <iostream>
11  using namespace std;
12
13  // forward declaration
14  class B;
15  class A {
16      private:
17        int numA;
18      public:
19        A(): numA(12) { }
20        // friend function declaration
21        friend int add(A, B);
22  };
23
24  class B {
25      private:
26         int numB;
27      public:
28         B(): numB(1) { }
29         // friend function declaration
30         friend int add(A , B);
31  };
32
33  // Function add() is the friend function of classes A and B
34  // that accesses the member variables numA and numB
35  int add(A objectA, B objectB)
36  {
37     return (objectA.numA + objectB.numB);
38  }
39
40  int main()
41  {
42      A objectA;
43      B objectB;
44      cout<<"Sum: "<< add(objectA, objectB);
45      return 0;
46  }
```

- A more meaningful use would to when you need to operate on objects of two different classes. That's when the friend function can be very helpful.
- You can definitely operate on two objects of different classes without using the friend function but the program will be long, complex and hard to understand.
- In this program, classes A and B have declared add() as a friend function. Thus, this function can access private data of both class.

Friend Functions

```
10  #include <iostream>
11  using namespace std;
12
13  // forward declaration
14  class B;
15  class A {
16      private:
17        int numA;
18      public:
19        A(): numA(12) { }
20        // friend function declaration
21        friend int add(A, B);
22  };
23
24  class B {
25      private:
26         int numB;
27      public:
28         B(): numB(1) { }
29         // friend function declaration
30         friend int add(A , B);
31  };
32
33  // Function add() is the friend function of classes A and B
34  // that accesses the member variables numA and numB
35  int add(A objectA, B objectB)
36  {
37     return (objectA.numA + objectB.numB);
38  }
39
40  int main()
41  {
42      A objectA;
43      B objectB;
44      cout<<"Sum: "<< add(objectA, objectB);
45      return 0;
46  }
```

- Here, add() function adds the private data numA and numB of two objects objectA and objectB, and returns it to the main function.
- To make this program work properly, a forward declaration of a class class B should be made as shown in the above example.
- This is because class B is referenced within the class A using code: friend int add(A , B);.

Friend Classes

```
... .. ...
class B;
class A
{
    // class B is a friend class of class A
    friend class B;
    ... .. ...
}

class B
{
    ... .. ...
}
```

- When a class is made a friend class, all the member functions of that class becomes friend functions.
- In this program, all member functions of class B will be friend functions of class A.
- Thus, any member function of class B can access the private and protected data of class A.
- But, member functions of class A cannot access the data of class B.
- Friend relation in C++ is only granted, not taken.
- Similarly, like a friend function, a class can also be made a friend of another class using keyword friend

Friend Classes

- Friend relation in C++ is only granted, not taken.
- For example, for class B to be a friend of class A, class A must explicitly declare that class B is its friend.
- Also, the friendship relation is neither symmetric nor transitive.
- For example, if class A is a friend of class B,and class B is a friend of class C, you cannot infer that class B is a friend of class A (again,friendship is not symmetric), that class C is a friend of class B (also because friendship is not symmetric), or that class A is a friend of class C (friendship is not transitive).

Introduction

- Every object has access to its own address through a pointer called **this** (a C++keyword).
- The **this** pointer is not part of the object itself and the memory occupied by the **this** pointer is not reflected in the result of a sizeof operation on the object.
- Rather,the **this** pointer is passed (by the compiler) as an implicit argument to each of the object's non-static member functions.
- Objects use the **this** pointer implicitly or explicitly to reference their data members and member functions.
- The type of the this pointer depends on the type of the object and whether the member function in which this is used is declared const.

Using the this Pointer

```
10  #include <iostream>
11  using namespace std;
12
13  class Test
14  {
15
16  public:
17      Test( int value= 0 ); // default constructor
18      void print() const;
19
20  private:
21      int x;
22  };
23
24  /// constructor
25  Test::Test( int value )
26      :x(value) // initialize xtovalue
27  {
28   // empty body
29  }
30
31  // print x using implicit and explicit this pointers
32  void Test::print() const
33  {
34      // implicitly use the this pointer to access the member x
35      cout << "x= " << x;
36
37      // explicitly use the this pointer and the arrow operator
38      // to access the member x
39      cout << "\n this->x =" << this->x;
40
41      // explicitly use the dereferenced this pointer and
42      // the dot operator to access the member x
43      cout << "\n(*this).x =" << (*this ).x << endl;
44  }
45
46  main()
47  {
48      Test testObject( 12 ); // instantiate and initialize testObject
49      testObject.print();
50  }
```

- Member function print (lines 32-44) first prints x by using the this pointer implicitly (line 35), only the name of the data member is specified.
- Then print uses two different notations to access x through the this pointer.
- The arrow operator ($->$) of the this pointer (line 39) and the dot operator (.) of the dereferenced this pointer(line 43).
- Note the parentheses around *this (line 43) when used with the dot member selection operator (.).
- The parentheses are required because the dot operator has higher precedence than the * operator.
- Without the parentheses,the expression *this.x would be evaluated as if it were parenthesized as *(this.x), which is a compilation error, because the dot operator cannot be used with a pointer.

Using the this Pointer

```
10  #include <iostream>
11  using namespace std;
12
13  class Box {
14  public:
15        // Constructor definition
16        Box(double l = 2.0, double b = 2.0, double h = 2.0) {
17           cout <<"Constructor called." << endl;
18           length = l;
19           breadth = b;
20           height = h;
21        }
22        double Volume() {
23           return length * breadth * height;
24        }
25        int compare(Box box) {
26           return this->Volume() > box.Volume();
27        }
28
29  private:
30        double length;     // Length of a box
31        double breadth;    // Breadth of a box
32        double height;     // Height of a box
33  };
34
35  int main(void) {
36     |
37     Box Box1(3.3, 1.2, 1.5);    // Declare box1
38     Box Box2(8.5, 6.0, 2.0);    // Declare box2
39
40     if(Box1.compare(Box2)) {
41        cout << "Box2 is smaller than Box1" <<endl;
42     } else {
43        cout << "Box2 is equal to or larger than Box1" <<endl;
44     }
45  }
```

- Notice that in line 40 Box1 object is called compare member function with Box2.
- When Box1 object calls the compare, in line 26 this pointer refers to Box1 object.
- As a result,

  this− >Volume() calls Volume member function for Box1 object.

  box.Volume() calls member function for Box2 object.

  return **this**− >Volume() > box.Volume() returns 1 if **this**− >Volume() is greater than box.Volume(). Otherwise the statement returns 0.

Using the this Pointer to Enable Cascaded Function Calls

```
12  // Time class definition
13  class Time
14  {
15  public:
16
17      /// Default Constructor
18      Time(int hour=0, int minute=0, int second=0);
19
20      /// Set hour, minute and second
21      Time &setTime(int,int,int);
22      /// Set hour (after validation)
23      Time &setHour(int);
24      /// Set minute (after validation)
25      Time &setMinute(int);
26      /// Set second (after validation)
27      Time &setSecond(int);
28
29      /// get functions (normally declared const)
30      /// Return hour
31      int getHour() const;
32      /// Return minute
33      int getMinute() const;
34      /// Return second
35      int getSecond() const;
36
37      /// print functions (normally declared const)
38      /// Print time in universal-time format
39      void printUniversal(void) const;
40      /// Print time in standard-time format (should be const)
41      void printStandard(void) const;
42
43  private:
44
45      int hour; // 0 - 23
46      int minute; // 0 - 59
47      int second; // 0 - 59
48  };
```

- Another use of the this pointer is to enable cascaded member-function calls.
- That is invoking multiple functions in the same statement.
- Notice that, each set function returns a reference in Time class type.
- This enables cascade function calls by using **this** pointer.
- Each of the set functions returns a reference at the same object that is called the set function.

Using the this Pointer to Enable Cascaded Function Calls

```
17  /// Time constructor initializes each data member to zero
18  Time::Time(int hour, int minute, int second )
19  {
20      setTime( hour, minute, second ); // validate and set time
21  }
22  /// Set new Time value using universal time
23  Time &Time::setTime(int h, int m, int s)
24  {
25      setHour(h);      // set private field hour
26      setMinute(m);    // set private field minute
27      setSecond(s);    // set private field second
28      return *this;
29  }
30  /// Set hour value
31  Time &Time::setHour(int h)
32  {
33      if (h >= 0 && h< 24 )
34          hour =h;
35      else
36          throw invalid_argument( "hour must be 0-23" );
37      return *this;
38  }
39  /// Set minute value
40  Time &Time::setMinute(int m)
41  {
42      if (m >= 0 && m< 60 )
43          minute =m;
44      else
45          throw invalid_argument( "minute must be 0-59" );
46      return *this;
47  }
48  /// Set second value
49  Time &Time::setSecond(int s)
50  {
51      if (s >= 0 && s< 60 )
52          second =s;
53      else
54          throw invalid_argument( "second must be 0-59" );
55      return *this;
56  }
```

```
58  /// Return hour value
59  int Time::getHour() const // get functions should be const
60  {
61      return hour;
62  }
63
64  /// Return minute value
65  int Time::getMinute() const
66  {
67      return minute;
68  }
69
70  /// Return second value
71  int Time::getSecond() const
72  {
73      return second;
74  }
75
76  /// print Time in universal-time format (HH:MM:SS)
77  void Time::printUniversal(void) const
78  {
79      cout << setfill('0') << setw( 2 ) << getHour() << ":"
80          << setw( 2 ) << getMinute() << ":" << setw( 2 ) << getSecond();
81  }
82
83  // print Time in standard-time format (HH:MM:SS AM or PM)
84  void Time::printStandard(void) const
85  {
86      if (getHour() == 0 || getHour() == 12 )
87          cout << 12;
88      else
89          cout << getHour() % 12;
90
91      cout << ":" << setfill('0') << setw( 2 ) << getMinute() << ":" << setw( 2 )
92          << getSecond();
93
94      if (getSecond() < 12)
95          cout << " AM";
96      else
97          cout << " PM";
98  }
```

Using the this Pointer to Enable Cascaded Function Calls

```
14  main()
15  {
16      Time t;     // create Time Object
17
18      // cascade function calls
19      t.setHour(18).setMinute(30).setSecond(22);
20
21      // output time in universal and standard formats
22      cout << "Universal time:";
23      t.printUniversal();
24
25      cout << "\nStandardtime: ";
26      t.printStandard();
27
28      cout << "\n\nNew standard time: ";
29
30      // cascaded function calls
31      t.setTime( 20, 20, 20 ).printStandard();
32      cout << endl;
33  }
```

- The program creates Time object t (line 16),then uses it in cascaded member-function calls (lines 19 and 31).
- Why does the technique of returning *this as a reference work?
- The dot operator(.) associates from left to right, so line 19 first evaluates t.setHour(18), then returns a reference to object t as the value of this function call.
- The remaining expression is then interpreted as
  t.setMinute(30).setSecond(22);
- The t.setMinute(30) call executes and returns a reference to the object t.
- The remaining expression is interpreted as
  t.setSecond(22);
- Line 31 also uses cascading. The calls must appear in the order shown in line 31,because printStandard as defined in the class does not return a reference to t.
- Placing the call to printStandard before the call to setTime in line 31 results in a compilation error.

Introduction

- There is an important exception to the rule that each object of a class has its own copy of all the data members of the class.
- In certain cases,only one copy of a variable should be shared by all objects of a class.
- A static data member is used for these and other reasons.
- Such a variable represents "class-wide" information (i.e.,a property that is shared by all instances and is not specific to any one object of the class).

Motivation

- Let's further motivate the need for static class-wide data with an example.
- Suppose that we have a video game with Martians and other space creatures.
- Each Martian tends to be brave and willing to attack other space creatures when the Martian is aware that there are at least five Martians present.
- If fewer than five are present, each Martian becomes cowardly.
- So each Martian needs to know the martianCount.
- We could endow each instance of class Martian with martianCount as a data member.
- If we do, every Martian will have a separate copy of the data member.
- Every time we create a new Martian, we'll have to update the data member martianCount in all Martian objects.

Motivation

- Doing this would require every Martian object to have, or have access to, handles to all other Martian objects in memory.
- This wastes space with the redundant copies and wastes time in updating the separate copies.
- Instead, we declare martianCount to be static. This makes martianCount class-wide data.
- Every Martian can access martianCount as if it were a data member of the Martian, but only one copy of the static variable martianCount is maintained by C++.
- This saves space. We save time by having the Martian constructor increment static variable martianCount and having the Martian destructor decrement martianCount.
- Because there's only one copy, we do not have to increment or decrement separate copies of martianCount for each Martian object.

Scope and Initialization of static Data Members

- Although they may seem like global variables,a class's static data members have class scope.
- Also, static members can be declared public, private or protected.
- A fundamental-type static data member is initialized by default to 0.
- If you want a different initial value, a static data member can be initialized once.
- A static const data member of int or enum type can be initialized in its declaration in the class definition.
- However,all other static data members must be defined at global namespace scope (i.e., outside the body of the class definition) and can be initialized only in those definitions.
- If a static data member is an object of a class that provides a default constructor, the static data member need not be initialized because its default constructor will be called.

Accessing static Data Members

- A class's private and protected static members are normally accessed through the class's public member functions or friends.
- A class's static members exist even when no objects of that class exist.
- To access a public static class member when no objects of the class exist,simply prefix the class name and the scope resolution operator(::) to the name of the data member.
- To access a private or protected static class member when no objects of the class exist, provide a public static member function and call the function by prefixing its name with the class name and scopere solution operator.
- A static member function is a service of the class, not of a specific object of the class.

Employee Class Interface

```
12  #include <string>
13  using namespace std;
14
15  // Employee class definition
16  class Employee
17  {
18  public:
19
20      /// Default Constructor
21      Employee(const string &first, const string &last);
22      /// Destructor
23      ~Employee();
24      /// return first name
25      string getFirstName() const;
26      /// return last name
27      string getLastName() const;
28      // return number of objects instantiated
29      static int getCount(); // static member function
30  private:
31
32      ///
33      string firstName;
34      ///
35      string lastName;
36      /// static data
37      static int count; // number of objects instantiated
38  };
```

- Employee.h includes a private static data member called count (line 37) and a public static member function called getCount (line 29).
- Also Class Employee's definition contains private data members firstName and lastName.
- The Employee constructor's header specifies that the constructor has two parameters(first and last).
- These parameters are passed via member initializer.
- Also Class Employee's definition contains const getFirstName and getLastName public member functions.

Employee Class Definition

```
 8  #include <iostream>
 9  #include "Employee.h"    // include definition of class Employee
10  using namespace std;
11
12  // define and initialize static data member at global namespace scope
13  int Employee::count = 0; // cannot include keyword static
14
15  // define static member function that returns number of
16  // Employee objects instantiated (declared staticinEmployee.h)
17  int Employee::getCount()
18  {
19      return count;
20  }
21
22  /// constructor uses member initializer list to pass initializer
23  /// values to constructors of member objects
24  Employee::Employee(const string &first, const string &last)
25      : firstName(first),          // initialize firstName
26        lastName(last)             // initialize lastName
27  {
28      ++count; // increment static count of employees
29
30      cout << "Employee constructor for "
31          << firstName<< " " << lastName << " called." << endl;
32  }
```

- Line 13 defines and initializes the data member count to zero at global namespace scope.
- Lines 17-20 define static member function getCount.
- Notice that neither line 13 nor line 17 includes keyword static, yet both lines refer to static class members.
- When static is applied to an item at global namespace scope, that item becomes known only in that file.

Employee Class Definition

```
 8  #include <iostream>
 9  #include "Employee.h"    // include definition of class Employee
10  using namespace std;
11
12  // define and initialize static data member at global namespace scope
13  int Employee::count = 0; // cannot include keyword static
14
15  // define static member function that returns number of
16  // Employee objects instantiated (declared staticinEmployee.h)
17  int Employee::getCount()
18  {
19      return count;
20  }
21
22  /// constructor uses member initializer list to pass initializer
23  /// values to constructors of member objects
24  Employee::Employee(const string &first, const string &last)
25        : firstName(first),          // initialize firstName
26          lastName(last)             // initialize lastName
27  {
28      ++count; // increment static count of employees
29
30      cout << "Employee constructor for "
31          << firstName<< " " << lastName << " called." << endl;
32  }
```

- When static is applied to an item at global namespace scope, that item becomes known only in that file.
- The static class members need to be available to any client code that uses the class, so we declare them static only in the .h file.
- Data member count maintains a count of the number of objects of class Employee that have been instantiated.

Employee Class Definition

```
22  /// constructor uses member initializer list to pass initializer
23  /// values to constructors of member objects
24  Employee::Employee(const string &first, const string &last)
25        : firstName(first),          // initialize firstName
26          lastName(last)             // initialize lastName
27  {
28      ++count; // increment static count of employees
29
30      cout << "Employee constructor for "
31          << firstName<< " " << lastName << " called." << endl;
32  }
33
34  /// output Employee object to show when its destructor is called
35  Employee::~Employee()
36  {
37      cout << "~Employee() called for "
38          << firstName<< " " << lastName << endl;
39
40      --count; // decrement static count of employees
41  }
```

- When objects of class Employee exist, member count can be referenced through any member function of an Employee object.
- count is referenced by both line 28 in the constructor and line 40 in the destructor.

Driver Program Employee Class

```
 8  #include <iostream>
 9  #include "Employee.h"    // include definition of class Employee
10  using namespace std;
11
12  main()
13  {
14      cout << "Number of employees before instantiation of any objects is "
15          << Employee::getCount() << endl;
16
17      {
18          Employee e1( "Susan", "Baker" );
19          Employee e2( "Robert", "Jones" );
20
21          // two objects exist;call static member function getCount again
22          // using the class name and the scope resolution operator
23          cout << "Number of employees afterobjects are instantiated is "
24              << Employee::getCount();
25
26          cout << "\n\nEmployee1: "
27              << e1.getFirstName() << " " << e1.getLastName()
28          << "\nEmployee 2: "
29              << e2.getFirstName() << " " << e2.getLastName() << "\n\n";
30      }
31
32      cout << "\nNumber of employees after objects are deleted is "
33          << Employee::getCount() << endl;
34  }
```

- The program uses static member function getCount to determine the number of Employee objects in memory at various points in the program.
- The program calls Employee::getCount() before any Employee objects have been created (line 15), after two Employee objects have been created (line 24) and after those Employee objects have been destroyed(line 33).

Driver Program Employee Class

```
 8  #include <iostream>
 9  #include "Employee.h"    // include definition of class Employee
10  using namespace std;
11
12  main()
13  {
14      cout << "Number of employees before instantiation of any objects is "
15          << Employee::getCount() << endl;
16
17      {
18          Employee e1( "Susan", "Baker" );
19          Employee e2( "Robert", "Jones" );
20
21          // two objects exist;call static member function getCount again
22          // using the class name and the scope resolution operator
23          cout << "Number of employees afterobjects are instantiated is "
24              << Employee::getCount();
25
26          cout << "\n\nEmployee1: "
27              << e1.getFirstName() << " " << e1.getLastName()
28          << "\nEmployee 2: "
29              << e2.getFirstName() << " " << e2.getLastName() << "\n\n";
30      }
31
32      cout << "\nNumber of employees after objects are deleted is "
33          << Employee::getCount() << endl;
34  }
```

- Lines 17-30 in main define a nested scope. Recall that local variables exist until the scope in which they're defined terminates.
- In this example, we create two Employee objects in lines 18-19 inside the nested scope.

Driver Program Employee Class

```
 8  #include <iostream>
 9  #include "Employee.h"    // include definition of class Employee
10  using namespace std;
11
12  main()
13  {
14      cout << "Number of employees before instantiation of any objects is "
15          << Employee::getCount() << endl;
16
17      {
18          Employee e1( "Susan", "Baker" );
19          Employee e2( "Robert", "Jones" );
20
21          // two objects exist;call static member function getCount again
22          // using the class name and the scope resolution operator
23          cout << "Number of employees afterobjects are instantiated is "
24              << Employee::getCount();
25
26          cout << "\n\nEmployee1: "
27              << e1.getFirstName() << " " << e1.getLastName()
28          << "\nEmployee 2: "
29              << e2.getFirstName() << " " << e2.getLastName() << "\n\n";
30      }
31
32      cout << "\nNumber of employees after objects are deleted is "
33          << Employee::getCount() << endl;
34  }
```

- As each constructor executes, it increments class Employee's static data member count.
- These Employee objects are destroyed when the program reaches line 30.
- At that point, each object's destructor executes and decrements class Employee's static data member count.

static Class Members

- A member function should be declared static if it does not access non-static data members or non-static member functions of the class.
- Unlike non-static member functions, a static member function does not have a **this** pointer,because static data members and static member functions exist independently of any objects of a class.
- The **this** pointer must refer to a specific object of the class,and when a static member function is called, there might not be any objects of its class in memory.
- Using the **this** pointer in a static member function is a compilation error.
- Declaring a static member function const is a compilation error. The const qualifier indicates that a function can not modify the contents of the object in which it operates, but static member functions exist and operate independently of any objects of the class.